

FINANCIAL MANAGEMENT SYSTEM

Submitted by

Ayesha Nikhat

Course

Data Science with GenAI

Institute

Fortune IT

Project Duration

6 Days

Software Used

Oracle SQL Command Line

Financial Management System using Oracle SQL

This project is developed to understand the real-time usage of Oracle SQL in financial and banking systems. In this project, different tables such as Customers, Accounts, Transactions, Loans, and Employees were created. Various operations such as data insertion, data cleaning, joins, aggregate functions, TCL commands, DCL commands, ALTER operations, and report generation were performed.

This project helped in improving practical SQL knowledge and understanding how financial data is managed in databases. The project also provided knowledge about relationships between tables, transaction management, and database security.

Introduction

The Financial Management System project is developed using Oracle SQL to understand real-time database operations used in banking and finance sectors. This project helps in managing customer details, account information, transaction records, loan information, and employee data in a structured database system.

The project covers important SQL concepts such as:

- DDL Commands
- DML Commands
- TCL Commands
- DCL Commands
- Data Cleaning
- Data Modeling
- Joins
- Aggregate Functions
- Constraints
- Report Generation

This project improved practical SQL knowledge and understanding of how financial data is stored, managed, updated, and secured in real-world database systems.

Objectives

- To understand database management concepts
- To learn Oracle SQL commands
- To create and manage tables
- To perform data cleaning operations

- To generate reports using joins and aggregate functions
- To understand transaction management using TCL commands
- To understand permission management using DCL commands
- To improve practical SQL skills

Database Tables Used

Table Name	Purpose
Customers	Stores customer information
Accounts	Stores account details
Transactions	Stores transaction records
Loans	Stores loan information
Employees	Stores employee information

Module 1 – Database Creation

Definition

Database creation is done using DDL (Data Definition Language) commands. Tables are used to store data in rows and columns.

Explanation

In this module, tables like Customers, Accounts, Transactions, Loans, and Employees were created using CREATE TABLE command. PRIMARY KEY and FOREIGN KEY were used to maintain relationships and proper data in the database. This module is the base of the entire project.

Execution

CREATING TABLES

- Customers
- Accounts
- Transactions
- Loans
- Employees

```
Run SQL Command Line
SQL> create table Customers (customer_id number primary key, customer_name varchar2(13), city varchar2(13), phone number(13));
Table created.
SQL> create table Accounts (account_id number primary key, customer_id NUMBER, account_type varchar2(13), balance NUMBER(10,2), foreign key (customer_id) REFERENCES Customers(customer_id));
Table created.
SQL> create table Transactions (transaction_id number primary key, account_id number(13), transaction_type varchar2(13), amount number(13), transaction_date DATE, foreign key(account_id) REFERENCES Accounts(account_id));
Table created.
SQL> create table Loans ( loan_id number PRIMARY KEY, customer_id number, loan_amount DECIMAL(10,2), loan_status varchar2(13), FOREIGN KEY (customer_id) REFERENCES Customers(customer_id));
Table created.
SQL> create table Employees (employee_id number PRIMARY KEY, employee_name varchar2(13), department varchar2(13), salary DECIMAL(10,2));
Table created.
SQL>
SQL>
SQL>
```

Module 2 – Data Modeling

Definition

Data modeling is the process of creating relationships between tables using primary keys and foreign keys.

Explanation

One customer can have multiple accounts and one account can have multiple transactions. Relationships are created using foreign keys.

Execution

1. One customer can have multiple accounts. How will you design the relationship?

Use One-to-Many Relationship.

One customer → Many accounts

Customer_ID is Primary Key in Customers table

Customer_ID is Foreign Key in Accounts table

2. How will you connect transactions with accounts?

Using Account_ID.

Account_ID is Primary Key in Accounts

Account_ID is Foreign Key in Transactions

3. Which columns should be Primary Keys?

Customers → Customer_ID

Accounts → Account_ID

Transactions → Transaction_ID

Loans → Loan_ID

Employees → Employee_ID

4. Which columns should not allow NULL values?

The following columns should have NOT NULL constraint:

Customer_Name

Account_Type

Employee_Name

Module 3 – Raw Data Insertion

Definition

Data insertion is done using INSERT command. It is used to add records into tables.

Explanation

Customer records, account details, transactions, employee details, and loan details were inserted into database tables. NULL values and invalid data were inserted intentionally for data cleaning practice.

Execution

- INSERTING CUSTOMERS VALUES
- INSERTING ACCOUNTS VALUES
- INSERTING TRANSACTIONS VALUES
- INSERTING EMPLOYEES VALUES
- INSERTING LOAN VALUES

```
Run SQL Command Line
SQL> INSERT INTO Customers VALUES (101, 'Ayesha', 'Hyderabad', 9876543210, 'ayasha@gmail.com');
1 row created.
SQL> INSERT INTO Customers VALUES (102, 'Ruhi', 'Mumbai', 9123456780, NULL);
1 row created.
SQL> INSERT INTO Customers VALUES (103, 'Shaista', 'Hyderabad', 9988776655, 'shaista@gmail.com');
1 row created.
SQL> INSERT INTO Customers VALUES (104, 'Arjun', 'Delhi', NULL, 'arjun@gmail.com');
1 row created.
SQL> INSERT INTO Customers VALUES (105, 'Sariya', 'Chennai', 9012345678, 'sariya@gmail.com');
1 row created.
SQL> INSERT INTO Customers VALUES (106, 'Meena', 'Pune', 9871234567, 'meena@gmail.com');
1 row created.
SQL> INSERT INTO Customers VALUES (107, 'Saba', NULL, 9765432109, 'saba@gmail.com');
1 row created.
SQL> INSERT INTO Customers VALUES (108, 'Divya', 'Bangalore', NULL, 'divya@gmail.com');
1 row created.
SQL> INSERT INTO Customers VALUES (109, 'Pooja', 'Kolkata', 9345678910, 'pooja@gmail.com');
1 row created.
SQL> INSERT INTO Customers VALUES (110, 'Aliya', NULL, 9234567891, 'aliya@gmail.com');
1 row created.
SQL>
```

```
1 row created.

SQL> INSERT INTO Transactions VALUES (301, 201, 'Deposit', 10000, TO_DATE('2026-05-10','YYYY-MM-DD'));

1 row created.

SQL> INSERT INTO Transactions VALUES (302, 202, 'Withdraw', 5000, TO_DATE('2026-05-11','YYYY-MM-DD'));

1 row created.

SQL> INSERT INTO Transactions VALUES (303, 203, 'Deposit', 7000, TO_DATE('2026-05-12','YYYY-MM-DD'));

1 row created.

SQL> INSERT INTO Transactions VALUES (304, 204, 'Withdraw', 8000, TO_DATE('2026-05-13','YYYY-MM-DD'));

1 row created.

SQL> INSERT INTO Transactions VALUES (305, 205, 'Deposit', 2000, TO_DATE('2026-05-14','YYYY-MM-DD'));

1 row created.

SQL> INSERT INTO Transactions VALUES (306, 206, 'Withdraw', 15000, TO_DATE('2026-05-15','YYYY-MM-DD'));

1 row created.

SQL> INSERT INTO Transactions VALUES (307, 207, 'Deposit', 3000, TO_DATE('2026-05-16','YYYY-MM-DD'));

1 row created.

SQL> INSERT INTO Transactions VALUES (308, 208, 'Withdraw', 9000, TO_DATE('2026-05-17','YYYY-MM-DD'));

1 row created.

SQL> INSERT INTO Transactions VALUES (309, 209, 'Deposit', 25000, TO_DATE('2026-05-18','YYYY-MM-DD'));

1 row created.

SQL> INSERT INTO Transactions VALUES (310, 210, 'Withdraw', 4000, TO_DATE('2026-05-19','YYYY-MM-DD'));

1 row created.
```

```
SQL>
SQL> INSERT INTO Employees VALUES (501, 'Ruh1', 'HR', 45000);

1 row created.

SQL> INSERT INTO Employees VALUES (502, 'Saba', 'Finance', 55000);

1 row created.

SQL> INSERT INTO Employees VALUES (503, 'Sariya', 'IT', 40000);

1 row created.

SQL> INSERT INTO Employees VALUES (504, 'Ayesha', 'Admin', 30000);

1 row created.

SQL> INSERT INTO Employees VALUES (505, 'Pooja', 'Sales', 35000);

1 row created.

SQL>
SQL> INSERT INTO Loans VALUES (401, 101, 250000, 'Approved');

1 row created.

SQL> INSERT INTO Loans VALUES (402, 102, 150000, 'Pending');

1 row created.

SQL> INSERT INTO Loans VALUES (403, 103, 200000, 'Rejected');

1 row created.

SQL> INSERT INTO Loans VALUES (404, 104, 300000, 'Approved');

1 row created.

SQL> INSERT INTO Loans VALUES (405, 105, 500000, 'Pending');

1 row created.
```

```
SQL> INSERT INTO Accounts VALUES (201, 101, 'Savings', 50000);

1 row created.

SQL> INSERT INTO Accounts VALUES (202, 102, 'Current', 75000);

1 row created.

SQL> INSERT INTO Accounts VALUES (203, 103, 'Current', 25000);

1 row created.

SQL> INSERT INTO Accounts VALUES (204, 104, 'Savings', 110000);

1 row created.

SQL> INSERT INTO Accounts VALUES (205, 105, 'Current', 15000);

1 row created.

SQL> INSERT INTO Accounts VALUES (206, 106, 'Savings', 82000);

1 row created.

SQL> INSERT INTO Accounts VALUES (207, 107, 'Current', 43000);

1 row created.

SQL> INSERT INTO Accounts VALUES (208, 108, 'Savings', 67000);

1 row created.

SQL> INSERT INTO Accounts VALUES (209, 109, 'Savings', 120000);

1 row created.

SQL> INSERT INTO Accounts VALUES (210, 110, 'Current', 35000);

1 row created.

SQL> █
```

.Module 4 – Data Cleaning

Definition

Data cleaning is the process of correcting invalid, duplicate, or missing data. Data cleaning removes invalid and missing values.

Explanation

In this module, NULL values were identified and updated. Negative balances and invalid records were removed. City names were also converted into uppercase for consistency.

Execution

- Find Customers with NULL Email Addresses
- Replace Missing Phone Numbers
- Remove Accounts Having Negative Balances
- Delete Invalid Transactions
- Identify Duplicate Customer Records
- Standardize City Names into Uppercase

```
Run SQL Command Line

SQL> select * from Customers;

CUSTOMER_ID CUSTOMER_NAME CITY          PHONE
-----
EMAIL
-----
101 Ayesha          HYDERABAD      9876543210
ayasha@gmail.com
102 Ruhi           MUMBAI         9123456780
103 Shaista        HYDERABAD      9988776655
shaista@gmail.com
CUSTOMER_ID CUSTOMER_NAME CITY          PHONE
-----
EMAIL
-----
104 Arjun          DELHI          9999999999
arjun@gmail.com
105 Saria          CHENNAI        9012345678
sariya@gmail.com
106 Meena          PUNE           9871234567
meena@gmail.com
CUSTOMER_ID CUSTOMER_NAME CITY          PHONE
-----
EMAIL
-----
107 Saba           BANGALORE      9765432109
Saba@gmail.com
108 Divya          BANGALORE      9999999999
divya@gmail.com
109 Pooja          KOLKATA        9345678910
pooja@gmail.com
```

```
Run SQL Command Line

SQL> SELECT * FROM Customers WHERE Email is NULL;

CUSTOMER_ID CUSTOMER_NAME CITY          PHONE
-----
EMAIL
-----
102 Ruhi           Mumbai         9123456780

SQL> UPDATE Customers SET Phone = 9999999999 WHERE Phone IS NULL;

2 rows updated.

SQL> DELETE FROM Accounts WHERE Balance < 0;

0 rows deleted.

SQL> SELECT Customer_Name, COUNT(*) FROM Customers GROUP BY Customer_Name HAVING COUNT(*) > 1;

no rows selected

SQL> UPDATE Customers SET City = UPPER(City);

10 rows updated.

SQL>
```


Module 5 – Data Manipulation

Definition

DML commands are used to insert, update, delete, and modify records inside tables.

Explanation

Account balances were updated, new customers were inserted, and inactive accounts were removed from the system.

Execution

- Update Account Balances After Transactions
- Add New Customers into the System
- Delete Inactive Customer Accounts
- Modify Customer Contact Details
- Increase Loan Amount for Selected Customers

```
Run SQL Command Line
1 row updated.
SQL> UPDATE Accounts
  2 SET Balance = Balance - 5000
  3 WHERE Account_ID = 202;
1 row updated.
SQL> select * from Accounts;
ACCOUNT_ID CUSTOMER_ID ACCOUNT_TYPE BALANCE
-----
201         101 Savings      60000
202         102 Current      70000
203         103 Current      25000
204         104 Savings     110000
205         105 Current      15000
206         106 Savings      82000
207         107 Current      43000
208         108 Savings      67000
209         109 Savings     120000
210         110 Current      35000
10 rows selected.
SQL> INSERT INTO Customers VALUES (111, 'Farah', 'Hyderabad', 9876501234, 'farah@gmail.com');
1 row created.
SQL> UPDATE Customers SET Phone = 9123456789, Email = 'ruhi@gmail.com' WHERE Customer_ID = 102;
1 row updated.
SQL> UPDATE Loans SET Loan_Amount = Loan_Amount + 5000 WHERE Customer_ID = 101;
1 row updated.
SQL> DELETE FROM Accounts WHERE Balance = 0;
0 rows deleted.
SQL>
```

Module 6 – SQL Queries & Reports

Definition

SQL queries are used to retrieve and analyze data from database tables.

Explanation

In this module, SELECT, WHERE, JOIN, and Aggregate Functions were used to display customer details, transaction history, and account reports from the database.

Execution

❖ Basic Queries

- Display All Customers
- Show All Account Details
- Find Customers from Hyderabad
- Display Accounts with Balance Greater than ₹50,000

❖ JOIN Queries

- Show Customer Name with Account Details
- Display Transaction History for Each Customer
- Find Customers Who Made Transactions Above ₹10,000
- Show Account Type-wise Total Balance

❖ Aggregate Functions

- Find Total Bank Balance
- Find Average Account Balance
- Find Maximum Transaction Amount
- Count Total Customers

```
Run SQL Command Line
SQL> SELECT * FROM Customers;
CUSTOMER_ID CUSTOMER_NAME CITY PHONE EMAIL
-----
101 Ayesha HYDERABAD 9876543210 ayesha@gmail.com
102 Ruhi MUMBAI 9123456789 ruhi@gmail.com
103 Shaista HYDERABAD 9988776655 shaista@gmail.com
104 Arjun DELHI 9999999999 arjun@gmail.com
105 Sarlya CHENNAI 9812345678 sarlya@gmail.com
106 Meena PUNE 9871234567 meena@gmail.com
107 Saba BANGALORE 9765432109 Saba@gmail.com
108 Divya KOLKATA 9999999999 divya@gmail.com
109 Pooja KOLKATA 9345678910 pooja@gmail.com
110 Aliya KOLKATA 9234567891 aliya@gmail.com
111 Farah Hyderabad 9876501234 farah@gmail.com
11 rows selected.

SQL> SELECT * FROM Accounts;
ACCOUNT_ID CUSTOMER_ID ACCOUNT_TYPE BALANCE
-----
201 101 Savings 60000
202 102 Current 70000
203 103 Current 25000
204 104 Savings 110000
205 105 Current 15000
206 106 Savings 82000
207 107 Current 43000
208 108 Savings 67000
209 109 Savings 120000
210 110 Current 35000
10 rows selected.

SQL> SELECT * FROM Customers WHERE City = 'Hyderabad';
CUSTOMER_ID CUSTOMER_NAME CITY PHONE EMAIL
-----
111 Farah Hyderabad 9876501234 farah@gmail.com
SQL> _
```

```

Run SQL Command Line
SQL>
SQL> SELECT C.customer_name, A.account_id, A.account_type, A.balance FROM Customers C INNER JOIN Accounts A ON C.customer_id = A.customer_id;

CUSTOMER_NAME ACCOUNT_ID ACCOUNT_TYPE BALANCE
-----
Ayesha 201 Savings 60000
Ruhi 202 Current 70000
Shaista 203 Current 25000
Arjun 204 Savings 110000
Sariya 205 Current 15000
Meena 206 Savings 82000
Saba 207 Current 43000
Divya 208 Savings 67000
Pooja 209 Savings 120000
Aliya 210 Current 35000

10 rows selected.

SQL> SELECT C.customer_name, T.transaction_id, T.transaction_type, T.amount, T.transaction_date FROM Customers C INNER JOIN Accounts A ON C.customer_id = A.customer_id
INNER JOIN Transactions T ON A.account_id = T.account_id;

CUSTOMER_NAME TRANSACTION_ID TRANSACTION_T AMOUNT TRANSACTI
-----
Ayesha 301 Deposit 10000 10-MAY-26
Ruhi 302 Withdraw 5000 11-MAY-26
Shaista 303 Deposit 7000 12-MAY-26
Arjun 304 Withdraw 8000 13-MAY-26
Sariya 305 Deposit 2000 14-MAY-26
Meena 306 Withdraw 15000 15-MAY-26
Saba 307 Deposit 3000 16-MAY-26
Divya 308 Withdraw 9000 17-MAY-26
Pooja 309 Deposit 25000 18-MAY-26
Aliya 310 Withdraw 4000 19-MAY-26
Ayesha 311 Deposit 12000 20-MAY-26

CUSTOMER_NAME TRANSACTION_ID TRANSACTION_T AMOUNT TRANSACTI
-----
Ruhi 312 Withdraw 3000 21-MAY-26
Shaista 313 Deposit 4500 22-MAY-26
Arjun 314 Withdraw 10000 23-MAY-26
Sariya 315 Deposit 6000 24-MAY-26
Meena 316 Withdraw 7000 25-MAY-26
Saba 317 Deposit 8500 26-MAY-26
Divya 318 Withdraw 2500 27-MAY-26

```

```

Run SQL Command Line
Sariya 315 Deposit 6000 24-MAY-26
Meena 316 Withdraw 7000 25-MAY-26
Saba 317 Deposit 8500 26-MAY-26
Divya 318 Withdraw 2500 27-MAY-26
Pooja 319 Deposit 18000 28-MAY-26
Aliya 320 Withdraw 5500 29-MAY-26

20 rows selected.

SQL> SELECT C.customer_name, T.amount FROM Customers C INNER JOIN Accounts A ON C.customer_id = A.customer_id INNER JOIN Transactions T ON A.account_id = T.account_id WHERE T.amount > 10000;

CUSTOMER_NAME AMOUNT
-----
Meena 15000
Pooja 25000
Ayesha 12000
Pooja 18000

SQL> SELECT account_type, SUM(balance) AS total_balance FROM Accounts GROUP BY account_type;

ACCOUNT_TYPE TOTAL_BALANCE
-----
Current 188000
Savings 439000

SQL> SELECT SUM(balance) AS total_bank_balance FROM Accounts;

TOTAL_BANK_BALANCE
-----
627000

SQL> SELECT AVG(balance) AS average_balance FROM Accounts;

AVERAGE_BALANCE
-----
62700

SQL> SELECT MAX(amount) AS maximum_transaction FROM Transactions;

MAXIMUM_TRANSACTION
-----
25000

```

Module 7 – TCL Commands

Definition

Transaction Control Language (TCL) commands are used to manage database transactions safely and securely.

Explanation

In this module, SAVEPOINT, ROLLBACK, and COMMIT commands were used to manage transactions safely during money transfer operations.

Execution

- Start a Transaction for Money Transfer
- Deduct Amount from One Account
- Add Amount into Another Account
- Create SAVEPOINT
- Rollback the Transaction
- Commit Successful Transaction

```
SQL> savepoint transfer_start;
```

Savepoint created.

```
SQL> UPDATE Accounts SET Balance = Balance - 5000 WHERE Account_ID = 201;
```

1 row updated.

```
SQL> UPDATE Accounts SET Balance = Balance + 5000 WHERE Account_ID = 202;
```

1 row updated.

```
SQL> SAVEPOINT transfer_point;
```

Savepoint created.

```
SQL> ROLLBACK TO transfer_point;
```

Rollback complete.

```
SQL> COMMIT;
```

Commit complete.

```
SQL>
```

Module 8 – DCL Commands

Definition

Data Control Language (DCL) commands are used to provide and remove permissions from database users.

Explanation

In this module, a new database user was created and permissions like SELECT and INSERT were granted using GRANT command. Permissions were removed using REVOKE command.

Execution

- **Create a New Database User**
- **Grant SELECT and INSERT Permissions**
- **Revoke DELETE Permissions**
- **Provide Limited Access to Finance Employees**

Run SQL Command Line

```
SQL> CREATE USER finance_user IDENTIFIED BY finance123;  
CREATE USER finance_user IDENTIFIED BY finance123  
*  
ERROR at line 1:  
ORA-01920: user name 'FINANCE_USER' conflicts with another user or role name  
  
SQL> GRANT SELECT, INSERT, DELETE ON Customers TO finance_user;  
Grant succeeded.  
  
SQL> REVOKE DELETE ON Customers FROM finance_user;  
Revoke succeeded.  
  
SQL> GRANT SELECT ON Accounts TO finance_user;  
Grant succeeded.  
  
SQL> GRANT SELECT, UPDATE ON Transactions TO finance_user;  
Grant succeeded.  
  
SQL> _
```

Module 9 – ALTER & DROP Operations

Definition

ALTER command is used to modify existing database tables, and DROP command is used to remove database objects permanently.

Explanation

In this module, new columns were added, column names were changed, and unnecessary tables were removed using ALTER and DROP commands.

Execution

- Add PAN Number Column to Customers Table
- Add Loan Status Column
- Rename a ColumnDrop Unnecessary Table

 Run SQL Command Line

```
SQL> ALTER TABLE Customers ADD PAN_Number VARCHAR2(20);
Table altered.

SQL> ALTER TABLE Loans ADD Loan_Status VARCHAR2(20);
Table altered.

SQL> ALTER TABLE Customers RENAME COLUMN Phone TO Mobile_Number;
Table altered.

SQL> DROP TABLE Temp_Data;
DROP TABLE Temp_Data
*
ERROR at line 1:
ORA-00942: table or view does not exist

SQL> DROP TABLE Employees;
Table dropped.

SQL> _
```

Conclusion

Through this project, basic knowledge about Oracle SQL and Financial Management System was learned. Different topics like table creation, data insertion, joins, data cleaning, data manipulation, and transaction management were understood clearly.

This project helped in understanding how customer details, account information, transaction records, loan details, and employee data are managed in a database system. Different SQL commands such as DDL, DML, TCL, and DCL were used to perform various database operations.

The project also improved SQL query writing skills and helped in understanding real-time database management used in banking and financial sectors. It provided practical knowledge about maintaining data, generating reports, handling transactions securely, and managing database security.

Advantages of Financial Management System

1. Helps in storing financial data in an organized way
2. Makes banking operations faster and easier
3. Reduces chances of data errors
4. Helps in quick access to customer information
5. Improves accuracy of financial records

THANK YOU

Thank you for giving me the opportunity to complete this project. This project helped in improving my SQL knowledge and understanding of database management concepts used in banking and financial systems.

Submitted By

Ayesha Nikhat

Course

Data Science with GenAI